

现代编程思想

哈希表与闭包

月兔公开课课程组

回顾

- 表
 - 键值对的集合，其中键不重复
 - 简单实现：二元组列表
 - 添加时向队首添加
 - 查询时从队首遍历
 - 树实现：二叉平衡树
 - 基于第五节课介绍的二叉平衡树，每个节点的数据为键值对
 - 对树操作时比较第一个参数

哈希函数

- 哈希函数/散列函数
 - 将任意长度的数据映射到某一固定长度的数据
 - 例如：MD5 算法可以把各种大小、各种格式的文件映射到 128 位的数据上
- 利用月兔的 `Hash` 特征，数据被映射到整数范围内

```
1. trait Hash { hash(Self) -> Int }  
2.  
3. test {  
4.     inspect("这是一个非常非常长的字符串".hash(), content="-735211761")  
5. }
```

哈希表

- 哈希表利用哈希函数，将数据映射到数组索引中
 - 进行快速的添加、查询、修改
 - 因为在现代计算机中，对于数组的随机访问是效率最高的操作

```
1. let array : Array[(K, V)] = ...
2. let key : K = ...
3. let value : V = ...
4.
5. // 键 --哈希--> 哈希值 --取模--> 数组索引
6. let index = key.hash().mod_u(array.length())
7.
8. array[index] = (key, value)           // 添加或更新
9. let (k, v) = array[index]             // 查询
```

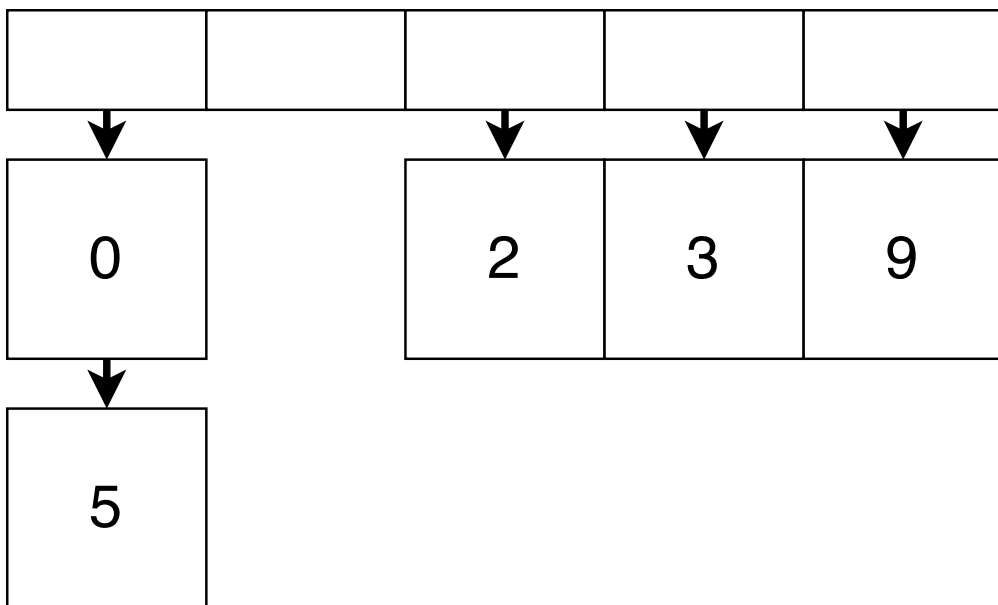
- 理想情况下，操作均为常数时间 $O(1)$
 - 相比之下，二叉平衡树操作均为对数时间 $O(\log n)$

哈希冲突

- 根据抽屉原理/鸽巢原理/生日问题
 - 不同数据的哈希可能相同
 - 不同的哈希映射为数组索引时可能相同
- 解决哈希表的冲突
 - 直接寻址（分离链接）：同一索引下用另一数据结构存储
 - 列表
 - 二叉平衡树等
 - 开放寻址
 - 线性探查：当发现冲突后，索引递增，直到查找空位放入
 - 二次探查（索引递增 1^2 2^2 3^2 ）等

哈希表：直接寻址

- 简单来说，就是定义一个列表的数组
- 当发生哈希/索引冲突时，将相同索引的数据装进一个列表中
- 添加数据时：计算哈希值及对应索引，将数据添加到索引对应的列表之中

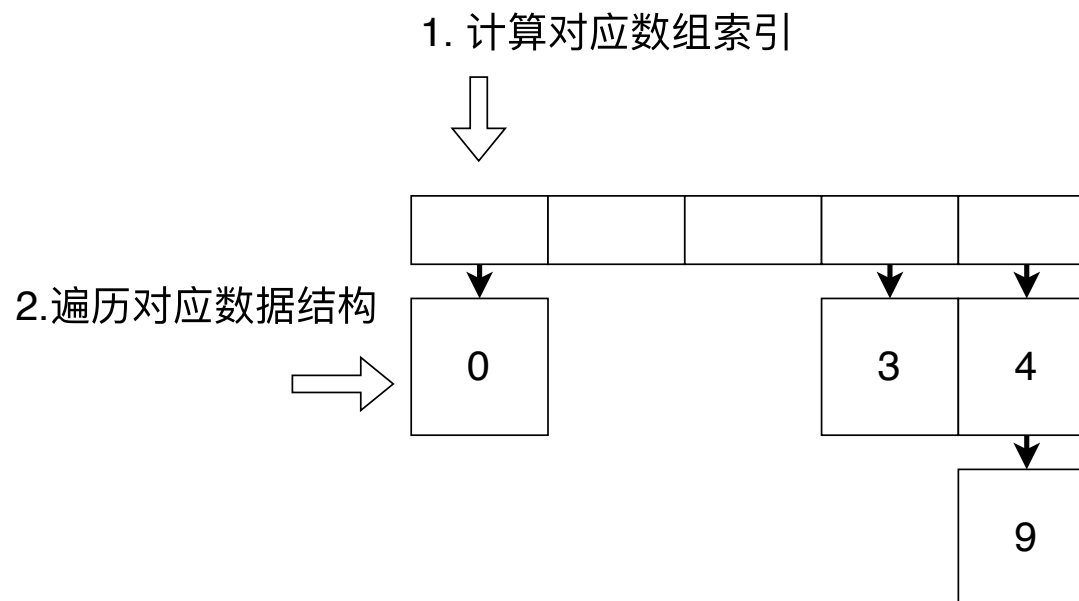


哈希表：直接寻址 - 数据结构

```
1. struct Entry[K, V] {
2.   key : K
3.   mut value : V // 允许原地更新值
4. }
5.
6. struct Bucket[V] {
7.   mut val : Option[(V, Bucket[V])] // None = 空列表, Some = (头元素, 剩余)
8. }
9.
10. struct HT_bucket[K, V] {
11.   mut values : Array[Bucket[Entry[K, V]]] // 存放列表的数组
12.   mut length : Int // 数组长度, 动态维护
13.   mut size : Int // 键值对数量, 动态维护
14. }
```

哈希表：直接寻址 - 添加/更新

- 添加/更新操作
 - 添加时，根据键的哈希计算出应当存放的位置
 - 遍历集合查找键
 - 如果找到，修改值
 - 否则，添加键值对
- 删除操作类同



哈希表：直接寻址 - 添加/更新代码

```
1. fn[K : Hash + Eq, V] HT_bucket::put(self : HT_bucket[K, V], key : K, value : V) -> Unit {
2.   let index = key.hash().mod_u(self.length) // 计算对应索引
3.   let mut bucket = self.values[index]       // 找到对应列表
4.   for { // 遍历列表查找键
5.     match bucket.val {
6.       Some((entry, rest)) => {
7.         if entry.key == key { entry.value = value; break } // 找到键, 更新值
8.         else { bucket = rest } // 继续查找
9.       }
10.      None => { // 未找到, 添加新键值对
11.        bucket.val = Some(({ key, value }, { val: None }))
12.        self.size = self.size + 1
13.        break
14.      }
15.    }
16.  }
17.  if self.size.to_double() / self.length.to_double() >= 0.75 {
18.    self.resize() // 根据负载判断是否需要扩容
19.  }
20. }
```

哈希表：负载与扩容

- 负载：键值对数量与数组长度的比值

```
1. load_factor = size / length
```

- 为什么需要扩容？
 - 负载上升 → 哈希冲突增加 → 链表变长
 - 操作时间增长，违背了哈希表的设计初衷
- 阈值设定的权衡
 - 阈值过高：链表过长，查找遍历时间增加
 - 阈值过低：频繁扩容，重新分配和重哈希的时间增加
 - 通常设置为 0.75

哈希表：直接寻址 - 删除

```
1. fn[K : Hash + Eq, V] HT_bucket::remove(self : HT_bucket[K, V], key : K) -> Unit {
2.     let index = key.hash().mod_u(self.length) // 计算对应索引
3.     let mut bucket = self.values[index]       // 找到对应列表
4.
5.     // 遍历列表查找键
6.     for {
7.         match bucket.val {
8.             Some((entry, rest)) => {
9.                 if entry.key == key {
10.                    // 找到键，删除（通过修改指针）
11.                    bucket.val = rest.val
12.                    self.size = self.size - 1
13.                    break
14.                } else {
15.                    bucket = rest // 继续查找
16.                }
17.            }
18.            None => break // 未找到，退出
19.        }
20.    }
21. }
```

- 直接寻址的查找、扩容操作留作练习

哈希表：开放寻址

- 线性探查：发现哈希冲突后，索引递增，直到查找到空位放入
- 不变式：键值对应当存放的位置与实际位置之间不存在空位
 - 为什么？否则确认键是否存在需遍历整个哈希表
 - 查找时遇到空位即可停止

键：0；哈希值：0；索引： $0 \equiv 0 \pmod{5}$

0				
---	--	--	--	--

键：1；哈希值：1；索引： $1 \equiv 1 \pmod{5}$

0	1			
---	---	--	--	--

键：5；哈希值：5；索引： $5 \equiv 0 \pmod{5}$

0	1	5		
---	---	---	--	--



哈希表：开放寻址 - 数据结构

```
1. struct Entry_open[K, V] {
2.   key : K
3.   mut value : V
4. } derive(Default)
5.
6. struct HT_open[K, V] {
7.   mut values : Array[Entry_open[K, V]] // 存放键值对的数组
8.   mut occupied : Array[Bool]           // 标记位置是否被占用
9.   mut length : Int                     // 数组长度，动态维护
10.  mut size : Int                       // 键值对数量，动态维护
11. }
```

- 我们采用基于默认值的数组
- 类似上节课展示的循环队列实现
- 基于 `Option` 的实现可自行尝试

哈希表：开放寻址 - 查找槽位

- 辅助函数：确定键是否存在
 - 如果存在，返回键的索引
 - 如果不存在，返回第一个空位的索引

```
1. fn[K : Hash + Eq, V] HT_open::find_slot(self : HT_open[K, V], key : K) -> Int {
2.   let hash = key.hash()
3.   let mut i = hash.mod_u(self.length)
4.
5.   while self.occupied[i] {
6.     if self.values[i].key == key {
7.       return i // 找到键
8.     } else {
9.       i = (i + 1).mod_u(self.length) // 线性探查
10.    }
11.  }
12.  return i // 找到空位
13. }
```

哈希表：开放寻址 - 添加/更新

```
1. fn[K : Hash + Eq + Default, V : Default] HT_open::put(  
2.     self : HT_open[K, V],  
3.     key : K,  
4.     value : V  
5. ) -> Unit {  
6.     let index = self.find_slot(key)  
7.     if self.occupied[index] { // 找到了对应的键, 更新值  
8.         self.values[index].value = value  
9.     } else { // 找到了空位, 插入新键值对  
10.         self.occupied[index] = true  
11.         self.values[index] = { key, value }  
12.         self.size = self.size + 1  
13.     }  
14.     // 检查是否需要扩容  
15.     if self.size.to_double() / self.length.to_double() >= 0.75 {  
16.         self.resize()  
17.     }  
18. }
```

开放寻址：删除的挑战

- 回顾：不变式
 - 键值对应当存放的位置与实际位置之间不存在空位
- 删除操作的问题：直接删除会破坏不变式

依次添加：0, 1, 5, 3

理想情况

0, 5	1		3	
------	---	--	---	--

实际情况

0	1	5	3	
---	---	---	---	--

删除 1 后

0		5	3	
---	--	---	---	--



方案1：“墓碑”

0	已删除	5	3	
---	-----	---	---	--

方案2：重新移动

0	5		3	
---	---	--	---	--

开放寻址：墓碑标记

```
1. enum Status {
2.     Empty
3.     Deleted    // 墓碑标记
4.     Occupied
5. }
6.
7. struct HT_open_tombstone[K, V] {
8.     mut values : Array[Entry_open[K, V]]
9.     mut occupied : Array[Status] // 改为使用 Status 而非 Bool
10.    mut length : Int
11.    mut size : Int
12. }
```

- 删除时：标记为 Deleted，而不是 Empty
- 插入时：Deleted 位置可以重用
- 查找时：遇到 Deleted 继续查找（视为有元素）；遇到 Empty 停止（确认不存在）

开放寻址：墓碑标记

```
1. fn[K : Hash + Eq, V] HT_open_tombstone::find_slot(  
2.   self : HT_open_tombstone[K, V],  
3.   key : K,  
4. ) -> Int {  
5.   let index = key.hash().mod_u(self.length)  
6.   loop (self.occupied[index], index, -1) { // 第一个空位默认为 -1  
7.     (Empty, i, -1) => i // 找到第一个空位  
8.     (Empty, _i, empty) => empty // 找到空位, 返回第一个空位  
9.     (Deleted, i, empty) => {  
10.      let new_i = (i + 1).mod_u(self.length)  
11.      let empty = if empty == -1 { empty } else { i } // 更新空位索引  
12.      continue (self.occupied[new_i], new_i, empty) // 继续查找  
13.    }  
14.    (Occupied, i, _empty) if self.values[i].key == key => i // 键存在  
15.    (Occupied, i, empty) => {  
16.      let new_i = (i + 1).mod_u(self.length)  
17.      continue (self.occupied[new_i], new_i, empty)  
18.    }  
19.  }  
20. }
```

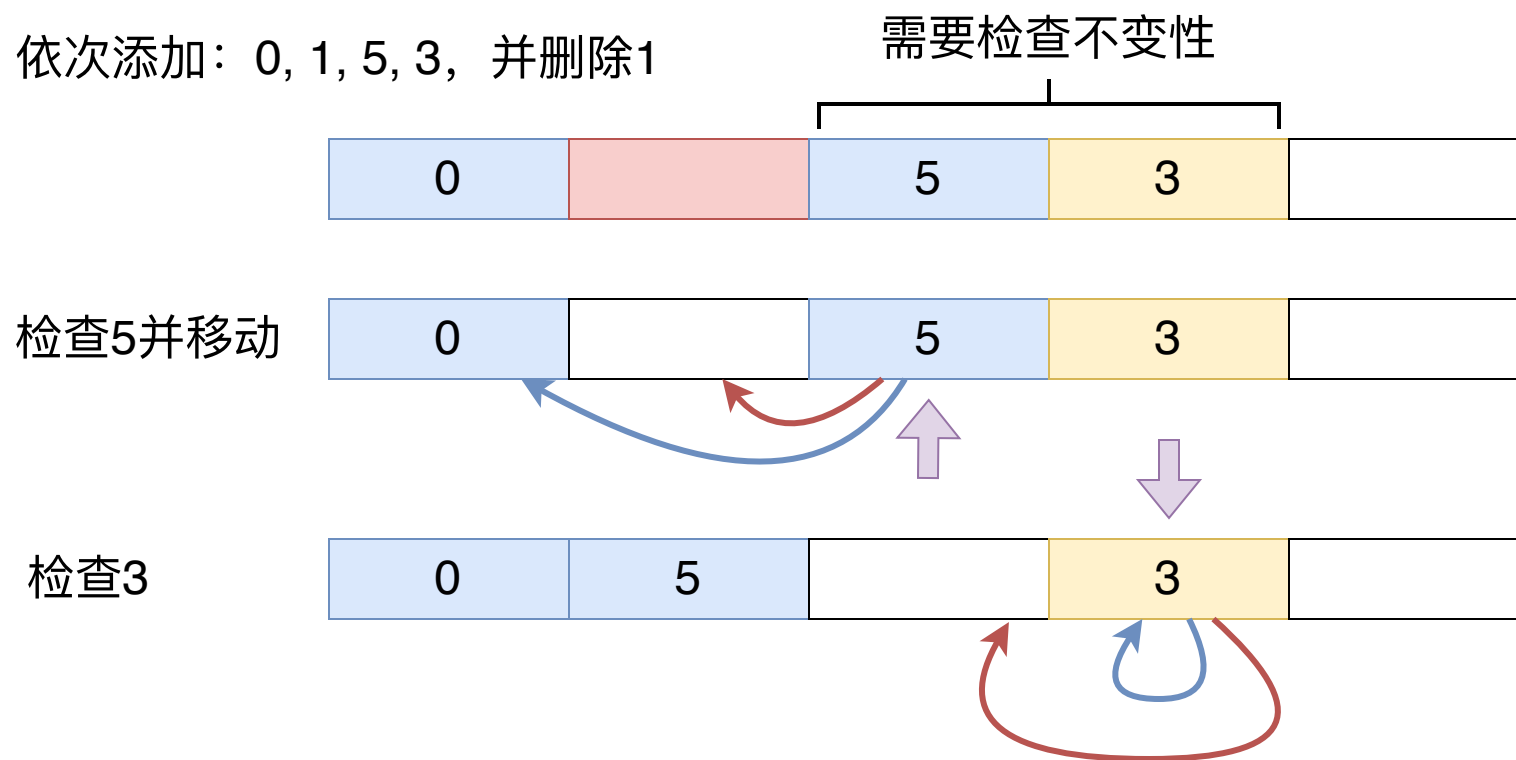
开放寻址：墓碑标记

```
1. fn [K : Hash + Eq + Default, V : Default] HT_open_tombstone::remove(  
2.     self : HT_open_tombstone[K, V],  
3.     key : K  
4. ) -> Unit {  
5.     let index = self.find_slot(key)  
6.     match self.occupied[index] {  
7.         Occupied => {  
8.             self.values[index] = Default::default()  
9.             self.occupied[index] = Deleted // 标记为已删除  
10.            self.size = self.size - 1  
11.        }  
12.        _ => () // 键不存在  
13.    }  
14. }
```

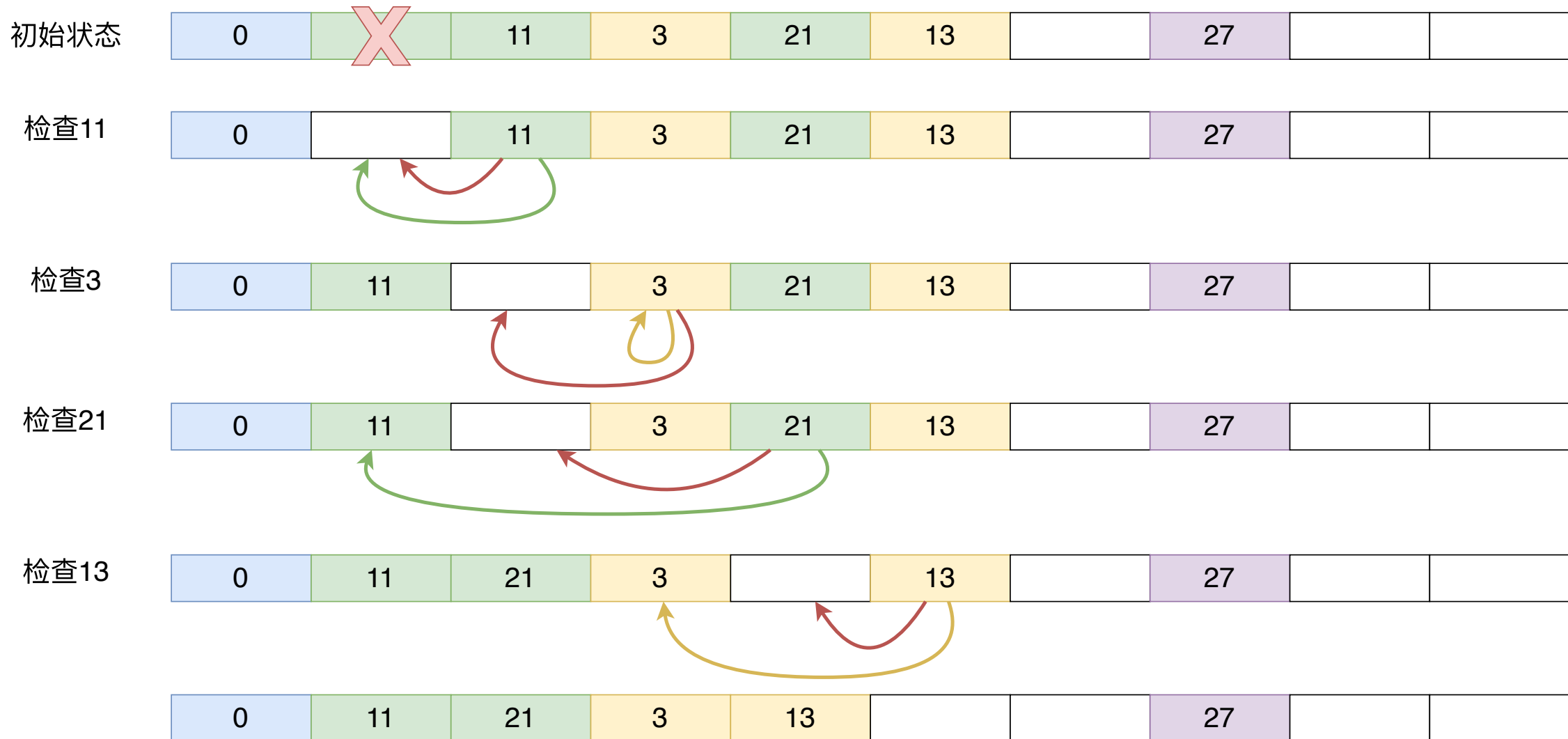
- 注意：多次添加删除后，会有大量 Deleted 标记
- 增加查询路径开销，需要定期重新整理

开放寻址：删除后压缩

- 开放寻址的另一种删除实现：删除后压缩
 - 我们通过移动哈希表中的元素来保持不变式（而不是通过标记）
 - 填补被删除元素留下的空位



哈希表：开放寻址



小结

- 哈希表
 - 哈希函数：将数据映射到固定长度
 - 哈希冲突：不同数据可能有相同的哈希值
- 两种哈希冲突解决方案
 - 直接寻址（分离链接）：使用列表存储冲突的键值对
 - 开放寻址（线性探查）：向后查找空位存储
- 开放寻址的删除操作
 - 墓碑标记
 - 删除后压缩